

PATRÓN DE DISEÑO: CADENA DE RESPONSABILIDAD

Capítulo 21 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El patrón Cadena de Responsabilidad (CoR) propone un bajo grado de acoplamiento entre el objeto que emite una solicitud y el conjunto de objetos potenciales que pueden atenderla.

Cuando existe más de un objeto capaz de manejar o resolver una solicitud de cliente, el patrón CoR recomienda dar a cada uno de estos objetos la oportunidad de procesar la solicitud en algún orden secuencial. Al aplicar el patrón CoR, cada uno de estos objetos manejadores potenciales puede organizarse en forma de cadena, donde cada objeto contiene una referencia al siguiente objeto en la cadena. El primer objeto de la cadena recibe la solicitud y decide si la maneja o la pasa al siguiente objeto. La solicitud fluye a través de los objetos de la cadena uno tras otro hasta que es atendida por alguno de ellos, o bien llega al final de la cadena sin ser procesada.

Como ejemplo, si $A \rightarrow B \rightarrow C$ son objetos capaces de manejar una solicitud en ese orden, entonces A debe manejar la solicitud o pasarla a B sin necesidad de conocer si B puede resolverla. Al recibir la solicitud, B decide manejarla o pasarla a C. Cuando C recibe la solicitud, debe manejarla o la solicitud cae fuera de la cadena sin ser procesada. En otras palabras, una solicitud enviada a la cadena de manejadores puede no ser atendida después de llegar al final de la misma.

A continuación se describen algunas de las características más importantes del patrón CoR:

- El conjunto de objetos manejadores potenciales y el orden en que forman la cadena pueden determinarse dinámicamente en tiempo de ejecución, dependiendo del estado actual de la aplicación.
- Un objeto cliente puede contar con diferentes conjuntos de manejadores para distintas solicitudes según su estado actual. Asimismo, un manejador determinado puede necesitar pasar una solicitud entrante a distintos manejadores dependiendo del tipo de solicitud y del estado de la aplicación cliente. Para que estas comunicaciones sean simples, todos los objetos manejadores potenciales deben ofrecer una interfaz consistente. En Java, esto puede lograrse haciendo que los distintos manejadores implementen una interfaz común o sean subclases de una clase abstracta padre común.
- El objeto cliente que inicia la solicitud, o cualquiera de los objetos manejadores que la reenvían, no necesitan conocer la identidad del objeto que recibirá la solicitud. Esto significa que ni el cliente ni ninguno de los manejadores en la cadena necesitan saber cuál de ellos atenderá efectivamente la solicitud.
- El procesamiento de la solicitud no está garantizado. Esto significa que la solicitud puede llegar al final de la cadena sin ser atendida. El ejemplo a continuación ilustra un escenario donde una solicitud de compra enviada a una cadena de manejadores no es aprobada ni siquiera al llegar al final de la cadena.

2. Ejemplo

Consideremos una aplicación para simular el proceso de aprobación de solicitudes de compra (SC) en una organización típica. En general, una SC debe ser autorizada por el representante gerencial correspondiente antes de que se pueda crear una orden a un proveedor.

Supongamos una organización con cuatro niveles gerenciales, listados en la Tabla 1, quienes pueden autorizar una SC cuyo monto sea inferior a su límite de autorización.

Tabla 1 — Niveles de Autorización de SC

Nivel Gerencial	Límite de Autorización
Gerente de Sucursal	\$25.000
Director Regional	\$100.000
Vicepresidente	\$200.000
Presidente y COO	\$400.000

Podemos definir diferentes clases (Listado 1) para representar cada nivel gerencial indicado en la Tabla 1.

A continuación se define la clase `SolicitudCompra` (Figura 1 y Listado 2), la cual representa una solicitud de compra.

Una determinada SC podría ser autorizada o gestionada por cualquiera de los representantes gerenciales listados en la Tabla 1. En otras palabras, cada una de las clases que representa los distintos niveles de gestión es un manejador potencial para la SC, por lo que no es aconsejable vincular una instancia de `SolicitudCompra` con alguno de los manejadores directamente.

Mediante el uso del patrón CoR, se logra un acoplamiento débil entre un objeto `SolicitudCompra` y el conjunto de objetos manejadores potenciales.

Aplicando el patrón CoR, definimos una clase abstracta `ManejadorSC` (Listado 3) que declara la interfaz común que deben ofrecer todos los manejadores potenciales (Figura 2).

Cada uno de los manejadores puede ahora rediseñarse como subclase de la clase abstracta `ManejadorSC` (Listado 4). Como parte de su implementación, cada manejador compara el monto de la SC con el límite de autorización del representante gerencial que representa. Si el monto de la SC es inferior al límite de autorización, el manejador autoriza la SC. En caso contrario, pasa la solicitud de autorización al siguiente manejador en la cadena.

Listado 1 — Clases que Representan los Distintos Niveles Gerenciales

```
class GerenteSucursal {
    static double LIMITE = 25000;
    ...
}

class DirectorRegional {
    static double LIMITE = 100000;
    ...
}

class Vicepresidente {
    static double LIMITE = 200000;
    ...
}
```

```
class PresidenteCOO {
    static double LIMITE = 400000;
    ...
}
```

Listado 2 — Clase SolicitudCompra

```
class SolicitudCompra {
    private int ID;
    private String descripcion;
    private double monto;

    public SolicitudCompra(int id, String desc, double amt) {
        ID = id;
        descripcion = desc;
        monto = amt;
    }

    public double getMonto() {
        return monto;
    }

    public String toString() {
        return ID + ":" + descripcion;
    }
}
```

Listado 3 — Clase Abstracta ManejadorSC

```
public abstract class ManejadorSC {
    private ManejadorSC siguienteManejador;
    private String nombreManejador;

    public ManejadorSC(String nombre) {
        nombreManejador = nombre;
    }

    public String getNombre() {
        return nombreManejador;
    }

    public abstract boolean autorizar(SolicitudCompra solicitud);

    public ManejadorSC getSiguienteManejador() {
        return siguienteManejador;
    }

    public void setSiguienteManejador(ManejadorSC manejador) {
        siguienteManejador = manejador;
    }
}
```

Para autorizar una SC, un cliente (Listado 5) debe:

1. Crear un conjunto de objetos manejadores potenciales de autorización de SC y organizarlos en orden ascendente por límite de autorización. Conectar cada manejador con el siguiente usando el método `setSiguienteManejador(ManejadorSC)`, lo que resulta en una cadena de manejadores potenciales de autorización de SC.
2. Enviar la solicitud de autorización de SC al primer objeto `ManejadorSC` invocando el método `autorizar` sobre dicho objeto, pasando la SC como argumento. Como puede observarse en la implementación de las subclases de `ManejadorSC`, una SC es autorizada si su monto es inferior al límite de autorización del manejador específico. En caso contrario, la solicitud se pasa al siguiente manejador potencial en la cadena. Una

vez que la SC es autorizada por uno de los manejadores, no se pasa al siguiente. La autorización de la SC no está garantizada en este ejemplo: si la solicitud llega al último manejador y el monto es superior a su límite de autorización, se muestra un mensaje apropiado y la SC permanece sin autorizar.

Al ejecutar el cliente GestorSC, se obtiene una salida similar a la siguiente:

```
Gerente de Sucursal Robin ha autorizado la SC - 1:Suministros de Oficina
VP Kate ha autorizado la SC - 2:Adquisición de Hardware
SC - 3:Campaña Publicitaria no pudo ser autorizada.
La Junta Directiva debe ser consultada para su aprobación.
Razón: Monto demasiado elevado
```

Listado 4 — Subclases Concretas de ManejadorSC

```
class GerenteSucursal extends ManejadorSC {
    static double LIMITE = 25000;
    public GerenteSucursal(String nombre) { super(nombre); }

    public boolean autorizar(SolicitudCompra solicitud) {
        double monto = solicitud.getMonto();
        if (monto <= LIMITE) {
            System.out.println(" Gerente de Sucursal " + getNombre()
                + " ha autorizado la SC - " + solicitud);
            return true;
        } else {
            return getSiguienteManejador().autorizar(solicitud);
        }
    }
}

class DirectorRegional extends ManejadorSC {
    static double LIMITE = 100000;
    public DirectorRegional(String nombre) { super(nombre); }

    public boolean autorizar(SolicitudCompra solicitud) {
        double monto = solicitud.getMonto();
        if (monto <= LIMITE) {
            System.out.println(" Director Regional " + getNombre()
                + " ha autorizado la SC - " + solicitud);
            return true;
        } else {
            return getSiguienteManejador().autorizar(solicitud);
        }
    }
}

class Vicepresidente extends ManejadorSC {
    static double LIMITE = 200000;
    public Vicepresidente(String nombre) { super(nombre); }

    public boolean autorizar(SolicitudCompra solicitud) {
        double monto = solicitud.getMonto();
        if (monto <= LIMITE) {
            System.out.println(" VP " + getNombre()
                + " ha autorizado la SC - " + solicitud);
            return true;
        } else {
            return getSiguienteManejador().autorizar(solicitud);
        }
    }
}

class PresidenteCOO extends ManejadorSC {
```

```

static double LIMITE = 400000;
public PresidenteCOO(String nombre) { super(nombre); }

public boolean autorizar(SolicitudCompra solicitud) {
    double monto = solicitud.getMonto();
    if (monto <= LIMITE) {
        System.out.println(" Presidente & COO " + getNombre()
            + " ha autorizado la SC - " + solicitud);
        return true;
    } else {
        System.out.println("SC - " + solicitud
            + " no pudo ser autorizada.\n"
            + "La Junta Directiva debe ser consultada.\n"
            + "Razón: Monto demasiado elevado");
        return false;
    }
}
}

```

Listado 5 — Clase Cliente GestorSC

```

public class GestorSC {
    private GerenteSucursal gerenteSucursal;
    private DirectorRegional directorRegional;
    private Vicepresidente vicepresidente;
    private PresidenteCOO coo;

    public static void main(String[] args) {
        GestorSC gestor = new GestorSC();
        gestor.crearFlujoAutorizacion();

        SolicitudCompra sc = new SolicitudCompra(1, "Suministros de Oficina",
1000);
        gestor.gerenteSucursal.autorizar(sc);

        sc = new SolicitudCompra(2, "Adquisición de Hardware", 175000);
        gestor.gerenteSucursal.autorizar(sc);

        sc = new SolicitudCompra(3, "Campaña Publicitaria", 600000);
        gestor.gerenteSucursal.autorizar(sc);
    }

    public void crearFlujoAutorizacion() {
        gerenteSucursal = new GerenteSucursal("Robin");
        directorRegional = new DirectorRegional("Oscar");
        vicepresidente = new Vicepresidente("Kate");
        coo = new PresidenteCOO("Drew");

        gerenteSucursal.setSiguienteManejador(directorRegional);
        directorRegional.setSiguienteManejador(vicepresidente);
        vicepresidente.setSiguienteManejador(coo);
    }
}

```

El diagrama de secuencia de la Figura 3 muestra el flujo de mensajes cuando se envía una solicitud de autorización de SC a la cadena de objetos manejadores potenciales.

3. Preguntas de Práctica

1. En el ejemplo anterior, una SC siempre debe ser aprobada por una sola persona en la cadena cuyo límite de aprobación sea superior al monto de la SC. Por ejemplo, una SC por \$50.000 requiere la aprobación del director regional (con límite de aprobación de \$100.000), pero no la del gerente de sucursal (con límite de \$25.000). En general, si una SC es aprobada por una persona en la cadena, no necesita la aprobación de ninguna otra. Sin embargo, en algunos casos puede requerirse que la SC sea aprobada por todos los individuos cuyo límite de aprobación sea inferior al monto de la SC, hasta que sea aprobada por aquel cuyo límite sea superior. Modifique la aplicación de ejemplo para implementar este proceso de aprobación de solicitudes de compra.

2. Consideremos un ISP (proveedor de servicios de Internet) con tres niveles de soporte técnico:

- a. Nivel de Servicio I (Básico) — Orientado a resolver problemas básicos de conectividad, como contraseñas incorrectas u olvidadas, número de marcación incorrecto, etc.
- b. Nivel de Servicio II — Cuando el equipo de Soporte Básico del Nivel I no puede resolver un problema, este se escala al equipo del Nivel II. Para la resolución del problema, el equipo del Nivel II asume que el usuario tiene un buen conocimiento de conceptos informáticos.
- c. Nivel de Servicio III — Cuando los equipos de los Niveles I y II no pueden resolver un problema, este se escala al equipo del Nivel III. Un técnico programa una visita al sitio del usuario para la resolución del problema.

Cree una aplicación utilizando el patrón CoR para simular la estructura de soporte técnico descrita anteriormente.

3. Consideremos una consultora de TI con tres niveles de coordinadores de recursos:

- Gestor de recursos local
- Coordinador de recursos regional
- Director de recursos corporativo

Cada vez que un consultor queda disponible, su perfil se envía al gestor de recursos local para verificar si existe algún requerimiento que coincida con sus habilidades. Si no existe ninguna coincidencia, los datos del consultor se envían al coordinador de recursos regional.

El coordinador regional, con acceso a un conjunto más amplio de requerimientos actuales y potenciales dentro de la región, intentará encontrar una posible coincidencia con las habilidades del consultor. Si no existe ninguna coincidencia, los datos del consultor se envían al director de recursos corporativo.

El director de recursos corporativo podrá buscar una coincidencia con los requerimientos del consultor en todas las regiones de la empresa.

Cree una aplicación utilizando el patrón CoR para simular este proceso.